JUnit 5 Beginner Guide

Learning the JUnit5 project from the greeting example

Project: PMIS-CapGemini/JUnit5
This guide explains testing, the project folders, Maven, POM files, assertions, and how to run the code.

1. What is testing?

Suppose you write this method:

```
public String sayHello(String name) {
    return "Hello, " + name + "!";
}
```

You expect that input `"Ada"` produces output `"Hello, Ada!"`. A test is a small Java program that checks whether this expectation is true.

Testing helps you detect bugs early, confirm that changes did not break old behavior, document what code should do, and verify many inputs automatically.

A test does not prove that every possible input is correct. It proves that the specific scenarios covered by the test behaved as expected.

2. Project structure

```
JUnit5
├── pom.xml
├── 01-first-test
├── 02-assertions
├── 03-lifecycle
├── 04-parameterized-tests
├── 05-nested-tests
└── 06-advanced-tests
```

Each numbered folder is a Maven module. Inside each module:

```
src/main/java   production/application code
src/test/java   test code
```

Production code contains the class being tested. Test code calls that class and checks its behavior.

Folder	Main idea
01-first-test	First test, @Test, assertEquals
02-assertions	Different assertion types
03-lifecycle	Test setup and cleanup
04-parameterized-tests	One test with many inputs
05-nested-tests	Organizing scenarios
06-advanced-tests	Repeated, dynamic, tagged, timeout tests

3. The greeting example

Greeting.java is production code:

```
package learning.junit5.first;

public class Greeting {
    public String sayHello(String name) {
        return "Hello, " + name + "!";
    }
}
```

`package` organizes classes. `public class Greeting` defines a class. The method returns a `String` and receives a `String name`. For the input `"Ada"`, Java joins the pieces and returns `"Hello, Ada!"`.

4. The greeting test

```
package learning.junit5.first;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class GreetingTest {
    @Test
    void saysHelloToThePerson() {
        Greeting greeting = new Greeting();

        String result = greeting.sayHello("Ada");

        assertEquals("Hello, Ada!", result);
    }
}
```

`@Test` tells JUnit that the following method is a test. The static import lets the test call `assertEquals` directly.

5. Arrange, Act, Assert

Most unit tests follow three steps:

1. **Arrange:** prepare objects and input.
2. **Act:** call the method being tested.
3. **Assert:** compare the actual result with the expected result.

```
Greeting greeting = new Greeting();       // Arrange
String result = greeting.sayHello("Ada"); // Act
assertEquals("Hello, Ada!", result);      // Assert
```

In plain English: create a greeting object, greet Ada, and verify the returned text is exactly `"Hello, Ada!"`.

6. What are assertions?

An assertion is a question asked by a test. For example:

```
assertEquals(expected, actual);
```

This asks: “Is the actual value equal to the expected value?” If yes, the test passes. If no, the test fails and JUnit reports the expected value, actual value, and location of the failure.

Common assertions

- `assertEquals(5, actual)` checks exact equality.
- `assertTrue(condition)` checks that a condition is true.
- `assertFalse(condition)` checks that a condition is false.
- `assertAll(...)` runs several related checks and reports their failures together.
- `assertThrows(Exception.class, action)` checks that an operation throws the expected exception.

```
IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> calculator.divide(10, 0)
);
assertEquals("Divisor cannot be zero", exception.getMessage());
```

The lambda `() -> calculator.divide(10, 0)` means “run this code when JUnit checks the assertion.”

7. What is Maven?

Maven is a Java build and project-management tool. It can compile source files, download libraries, compile tests, run tests, and generate reports.

Maven is configured using `pom.xml`. POM means **Project Object Model**.

The parent POM defines the project identity, Java version, JUnit dependencies, and six child modules. The important parts are:

```
<modules>
    <module>01-first-test</module>
    ...
    <module>06-advanced-tests</module>
</modules>

<maven.compiler.release>17</maven.compiler.release>
<junit.version>5.10.2</junit.version>
```

JUnit is declared with test scope. That means it is needed to compile and run tests, but it is not part of the production application.

Maven Surefire connects Maven to JUnit:

```
Maven → Surefire → JUnit Platform → JUnit Jupiter
       → discover tests → execute tests → report results
```

8. How to run the project

Open the **JUnit5** folder itself in VS Code:

```
C:\Users\mufgb\Downloads\PMIS_CapG\Java\PMIS-CapGemini\JUnit5
```

Open a terminal in that folder and check installations:

```
java --version
mvn --version
```

Run all modules:

```
mvn test
```

Run only the first module:

```
mvn -pl 01-first-test test
```

Run one test class:

```
mvn -pl 01-first-test -Dtest=GreetingTest test
```

Run one method:

```
mvn -pl 02-assertions -Dtest=CalculatorTest#addsTwoNumbers test
```

Always run Maven from the folder containing the relevant `pom.xml`. Running it from the higher `Java` folder gives “there is no POM in this directory.”

9. What happens when a test fails?

Change the greeting expectation temporarily:

```
assertEquals("Hi, Ada!", result);
```

Then run:

```
mvn -pl 01-first-test test
```

The test fails because the expected value is `"Hi, Ada!"`, but the actual production result is `"Hello, Ada!"`.

Do not change the test only to make it green. Ask whether the production code is wrong or whether the expected behavior is wrong. Restore the correct value after this demonstration.

10. Lifecycle tests

The lifecycle example uses a shopping cart and these annotations:

- `@BeforeAll`: once before all tests in the class.
- `@BeforeEach`: before every test; useful for fresh test data.
- `@AfterEach`: after every test; useful for cleanup.
- `@AfterAll`: once after all tests in the class.

```
@BeforeAll
  @BeforeEach → @Test → @AfterEach
```

```
  @BeforeEach → @Test → @AfterEach
@AfterAll
```

A fresh cart prevents one test from accidentally depending on another test.

11. Parameterized tests

A parameterized test runs the same test body with different inputs.

```
@ParameterizedTest
@ValueSource(strings = {"", " ", "   "})
@NullSource
void recognizesBlankText(String input) {
    assertTrue(textUtils.isBlank(input));
}
```

This runs with an empty string, spaces, and null. CSV data can provide several arguments:

```
@CsvSource({"A, 3, AAA", "go, 2, gogo", "Java, 1, Java"})
void repeatsText(String text, int count, String expected) { ... }
```

12. Nested tests

`@Nested` groups tests around a scenario. One group can describe valid usernames and another invalid usernames. `@DisplayName` makes the IDE and report easier to read without changing Java method names.

```
Username validation
├── when the username is valid
│   └── accepts letters and numbers
└── when the username is invalid
    ├── rejects short names
    └── rejects special characters
```

13. Advanced tests

- `@RepeatedTest(3)` runs a test three times.
- `@TestFactory` creates dynamic tests at runtime.
- `@Timeout(1)` fails a test that takes too long.
- `@Tag("advanced")` labels tests for later selection.

14. Unit testing versus other testing

Type	What it checks
Unit test	One small class or method. Fast and focused; usually no real database or network.
Integration test	Several parts working together, such as service plus database.
End-to-end test	A complete user workflow, such as login through checkout.

Your project starts with unit testing. Learn the basic pattern first:

```
Input → method → output
       ↓
test checks whether output matches expectation
```

15. Recommended study method

1. Run `mvn -pl 01-first-test test`.
2. Read the production class and test class side by side.
3. Identify Arrange, Act, and Assert.
4. Change the expected greeting and observe the failure.
5. Restore it, then study assertions.
6. Continue through lifecycle, parameterized, nested, and advanced tests.

Core idea: a unit test prepares input, calls one behavior, and checks the result against a clearly stated expectation.